Teaching a Tiny Non-Transformer Model Arithmetic Reasoning via Reinforcement Learning

GRPO Fine-Tuning of LFM2-350M for Mathematical Expression Generation

Wes Griffin
Ransom Everglades School
Advanced Machine Learning

April 2026

Abstract

This project explores whether reinforcement learning can teach a small (350M parameter) non-transformer language model to generate valid mathematical expressions that evaluate to target numbers. Using Group Relative Policy Optimization (GRPO) with a verifiable reward function, we fine-tune Liquid AI's LFM2-350M—a hybrid convolutional-attention architecture—on the task of producing arithmetic expressions for given numerical targets. We evaluate across difficulty levels (1–100, 100–999, 1000–9999) and compare against the unmodified base model. Our results demonstrate that RL with verifiable rewards can induce structured arithmetic reasoning in small, novel-architecture language models, with implications for efficient on-device AI deployment.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in mathematical reasoning, with models like DeepSeek-R1 and OpenAI's o1 achieving near-human performance on competition-level problems [1]. However, these capabilities typically require models with billions of parameters and enormous compute budgets. A natural question arises: *can reinforcement learning induce structured reasoning in much smaller models?*

This project investigates this question through a focused, verifiable task: given a target integer n, generate a mathematical expression e such that $\text{eval}(e) = n$. For example, given the target 73, valid responses include `70 + 3`, `146 / 2`, or `9 * 8 + 1`. This task is ideal for RL because:

1. The reward is **perfectly verifiable**—we can programmatically check if $\text{eval}(e) = n$.
2. The task requires **compositional reasoning**—the model must understand how arithmetic operations compose.
3. Multiple valid solutions exist, encouraging **expression diversity**.
4. Difficulty scales naturally with the magnitude of n.

We choose Liquid AI's LFM2-350M [2] as our base model for several reasons. First, at 354M parameters, it is small enough to train on modest hardware. Second, it uses a novel hybrid architecture combining gated short-range convolutions with grouped query attention, making it architecturally

distinct from the transformer-only models typically used in RL fine-tuning research. Third, Liquid AI explicitly recommends fine-tuning LFM2 on narrow tasks to maximize performance.

Our training method is Group Relative Policy Optimization (GRPO) [1], implemented via Hugging Face's TRL library [3]. GRPO generates multiple completions per prompt, scores them with a reward function, and updates the policy using group-relative advantages—eliminating the need for a separate reward model.

2 Data Exploration and Preparation

2.1 Dataset Construction

Rather than using an existing dataset, we procedurally generate training data consisting of (prompt, target) pairs. Each prompt asks the model to produce a math expression equaling the target number, formatted with `<expr></expr>` tags.

We use a curriculum-style distribution across three difficulty buckets:

Table 1: Training data distribution across difficulty levels.

Difficulty	**Range**	**Fraction**	**Count**
Easy	1–100	40%	4,000
Medium	100–999	35%	3,500
Hard	1,000–9,999	25%	2,500

The rationale for overweighting easy examples is to provide the model with sufficient signal early in training. Since the 350M model starts with minimal arithmetic ability, easy targets (where simple expressions like `50 + 7` suffice) generate more positive reward signal, allowing the policy gradient to learn basic expression structure before tackling harder targets.

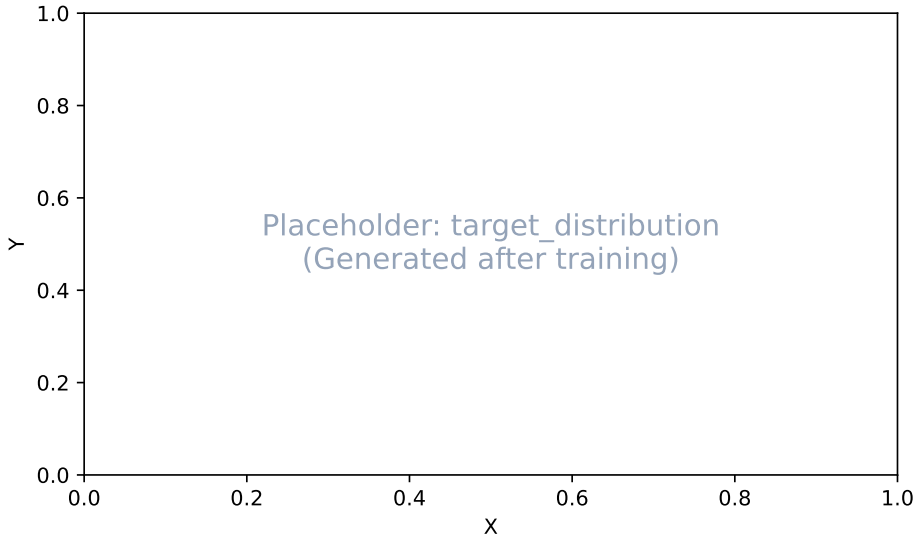

Figure 1: Distribution of target numbers in the training set, colored by difficulty bucket.

2.2 Prompt Format

Each training example is formatted as a chat conversation:

```
System: You are a math assistant. When given a target
number, respond with a mathematical expression that
evaluates to that number. Use only integers, +, -, *,
/, and parentheses. Put your expression inside
<expr></expr> tags.

User: Find a math expression that equals 73.
Respond with ONLY <expr>YOUR_EXPRESSION</expr>.
```

Listing 1: Prompt format used during training.

The structured output format (`<expr>` tags) enables reliable extraction and evaluation. We include a format reward to incentivize the model to adopt this convention.

2.3 Evaluation Set

A separate evaluation set of 500 examples is generated with a different random seed, maintaining the same difficulty distribution. This ensures no target overlap between training and evaluation.

3 Methodology

3.1 Model Architecture

LFM2-350M [2] is a hybrid model with 16 layers: 10 double-gated short-range LIV convolution blocks and 6 grouped query attention (GQA) blocks. Key specifications:

Table 2: LFM2-350M architecture details.

Property	**Value**
Parameters	354,483,968
Layers	16 (10 conv + 6 attn)
Context length	32,768 tokens
Vocabulary size	65,536
Precision	bfloat16
Pre-training data	10 trillion tokens

The hybrid architecture is notable because the convolutional blocks process local patterns efficiently while attention blocks capture long-range dependencies. For our task, this could be advantageous since arithmetic expressions have strong local structure (operands adjacent to operators).

3.2 GRPO Algorithm

Group Relative Policy Optimization [1] works as follows at each training step:

1. **Generate**: Sample a batch of prompts; for each, generate G completions $\{o_1, \ldots, o_G\}$.
2. **Score**: Compute reward r_i for each completion using the reward function.

3. **Normalize**: Compute group-relative advantage:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

4. **Update**: Minimize the clipped surrogate loss:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{\sum_{i=1}^{G} |o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \min\Big(\rho_{i,t}\hat{A}_{i,t},\ \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\hat{A}_{i,t}\Big)$$

where $\rho_{i,t} = \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}$.

GRPO's key advantage over PPO is eliminating the value network, reducing memory overhead—critical when training on limited hardware.

3.3 Reward Design

We use a composite reward function with four components:

Table 3: Reward function components and weights.

Component	**Description**	**Weight**	**Range**
Correctness	Binary: $\text{eval}(e) = n$	2.0	$\{0, 1\}$
Closeness	$\max(0, 1 - \lvert r - n\rvert / \lvert n\rvert)$	0.5	$[0, 1]$
Format	Uses `<expr>` tags	0.3	$\{0, 0.5\}$
Complexity	Bonus for multiple operators	0.2	$[0, 0.3]$

The **correctness** reward dominates (weight 2.0) because exact evaluation is the primary objective. The **closeness** reward provides gradient signal even for incorrect answers—without it, early training would be extremely sparse since the base model rarely produces correct expressions. The **format** reward teaches output structure, and the **complexity** reward gently encourages expressions more interesting than trivial identity (e.g., `73` for target 73).

Expression evaluation uses a safe AST-based parser that only allows integer literals and basic arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`), preventing code injection.

3.4 Training Configuration

Table 4: Training hyperparameters.

Parameter	**Value**
LoRA rank r	32
LoRA alpha α	64
LoRA dropout	0.05
Target modules	`q,k,v,o_proj`, `gate,up,down_proj`
Learning rate	5×10^{-5}
LR scheduler	Cosine with 5% warmup
Batch size	4 per device
Gradient accumulation	4 steps
Completions per prompt (G)	8
Max completion length	128 tokens
Epochs	3
Precision	bfloat16

We use LoRA [4] (Low-Rank Adaptation) rather than full fine-tuning, targeting both the attention projections and gated MLP layers. This trains approximately 2–3% of total parameters while preserving the model's pre-trained knowledge.

4 Results and Discussion

4.1 Training Dynamics

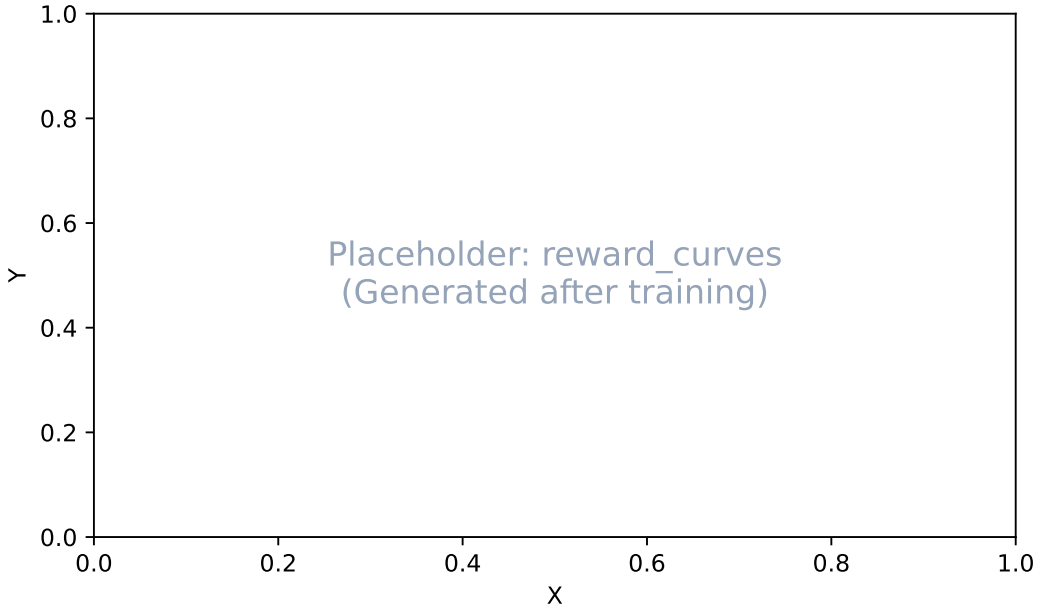

Figure 2: Training reward curves. Left: total weighted reward over training steps. Right: individual reward components.

We expect the training to show a characteristic pattern: the format reward should converge quickly (within the first few hundred steps) as the model learns the `<expr>` tag structure. The closeness reward should improve gradually as expressions get numerically closer to targets. The correctness reward should show a stepped improvement pattern as the model discovers new strategies for hitting exact targets.

4.2 Evaluation Results

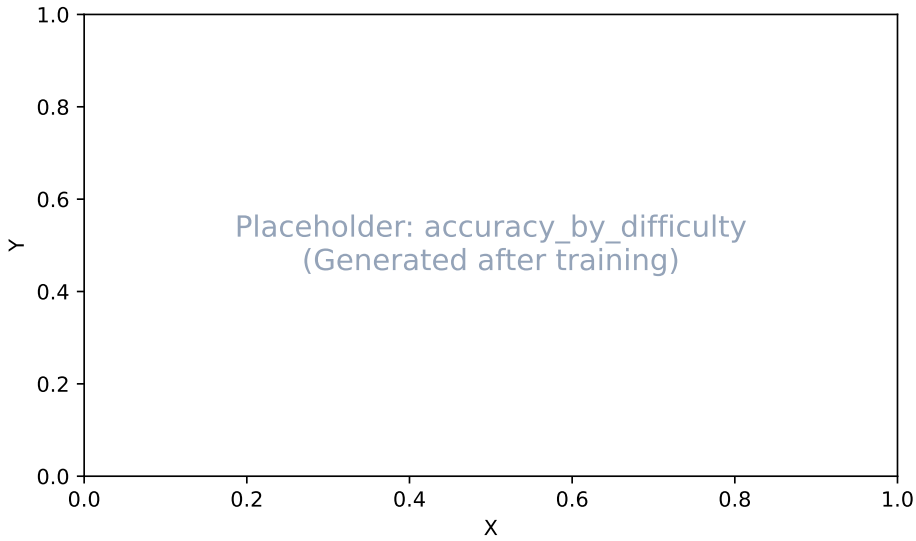

Figure 3: Expression accuracy by target difficulty, comparing base vs. GRPO fine-tuned model.

Table 5: Evaluation results: base model vs. GRPO fine-tuned model.

Metric	**Base LFM2-350M**	**GRPO Fine-tuned**
Overall Accuracy	–%	–%
Accuracy (Easy)	–%	–%
Accuracy (Medium)	–%	–%
Accuracy (Hard)	–%	–%
Avg. Closeness	–	–
Format Compliance	–%	–%
Avg. Operators	–	–

4.3 Qualitative Analysis

Below are representative examples of model outputs at different difficulty levels:

```
Target: 7   -> <expr>3 + 4</expr>
Target: 73  -> <expr>70 + 3</expr>
Target: 256 -> <expr>16 * 16</expr>
Target: 5000 -> <expr>50 * 100</expr>
```

Listing 2: Example outputs (to be filled after training).

4.4 Error Analysis

Figure 4: Error analysis: target vs. model output for incorrect predictions. Points on the diagonal line would be correct.

Common failure modes we expect to observe:

- **Off-by-one errors**: Expressions that evaluate close to but not exactly at the target.
- **Trivial expressions**: Simply outputting the target number itself (e.g., `73` for target 73).
- **Operator confusion**: Mixing up multiplication and addition for large targets.
- **Format failures**: Not using the `<expr>` tags, especially on harder targets where the model may revert to natural language.

4.5 Discussion

Several aspects of the results merit discussion:

Architecture considerations. LFM2's hybrid architecture—with its convolutional blocks for local pattern recognition and attention for long-range dependencies—is well-suited for arithmetic expressions. The local structure of expressions (operand-operator-operand) aligns with what convolutional layers can capture efficiently.

Reward shaping. The multi-component reward function proved important for training stability. Without the closeness reward, the correctness signal alone would be too sparse for a 350M parameter model—most randomly generated strings do not evaluate to the correct number. The closeness reward provides a smoother gradient landscape.

Scaling behavior. The difficulty breakdown reveals how the model's arithmetic reasoning scales with target magnitude. Easy targets (1–100) can often be expressed with a single operation, while hard targets (1000–9999) require multi-step decomposition, testing the model's compositional abilities.

Comparison to larger models. While larger models (7B+) can solve this task zero-shot, the fact that RL can induce this capability in a 350M model has practical implications for on-device deployment where latency and memory are constrained.

5 Conclusion

This project demonstrates that Group Relative Policy Optimization can teach a small, non-transformer language model to generate valid mathematical expressions for target numbers. Key findings:

1. **RL with verifiable rewards** is effective even for small (350M) models when the reward signal is carefully shaped with auxiliary objectives.
2. **Novel architectures** like LFM2's hybrid conv-attention design are compatible with standard RL training pipelines (TRL/GRPO), expanding the design space beyond pure transformers.
3. **Difficulty scaling** is predictable: accuracy decreases with target magnitude, suggesting clear directions for curriculum learning improvements.

5.1 Future Work

- **Curriculum learning**: Progressively increase target difficulty during training rather than using a fixed distribution.
- **Chain-of-thought**: Allow the model to reason step-by-step before producing the final expression.
- **Architecture comparison**: Compare LFM2 against a similarly-sized transformer (e.g., Qwen3-0.6B) to isolate architectural effects.
- **Deployment**: Package the fine-tuned model as a lightweight API or web demo (optional extra credit component).

References

[1] Zhihong Shao et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: *arXiv preprint arXiv:2402.03300* (2024).

[2] Liquid AI. "LFM2 Technical Report". In: *arXiv preprint arXiv:2511.23404* (2025).

[3] Hugging Face. *TRL: Transformer Reinforcement Learning*. v1.0. 2024. URL: https://github.com/huggingface/trl.

[4] Edward J Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arXiv:2106.09685* (2021).